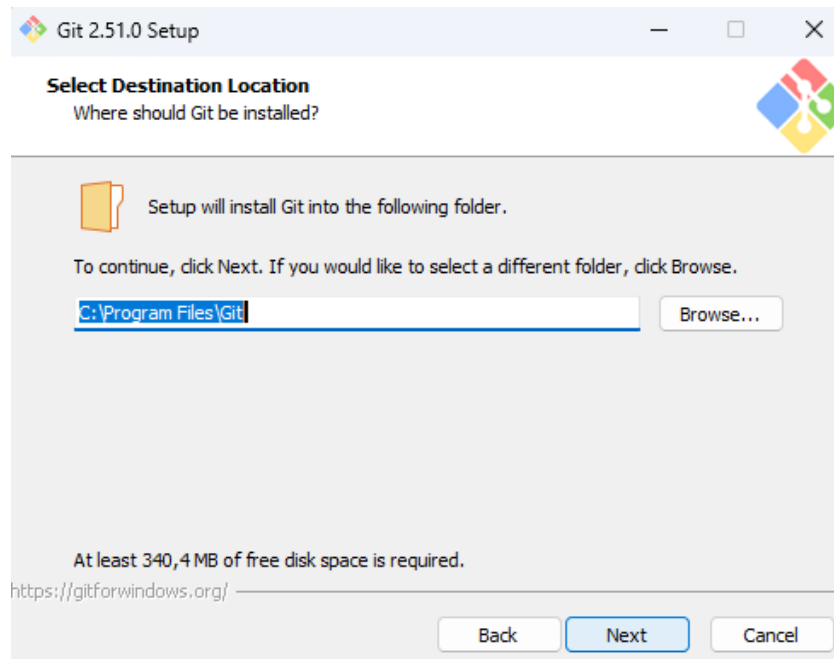


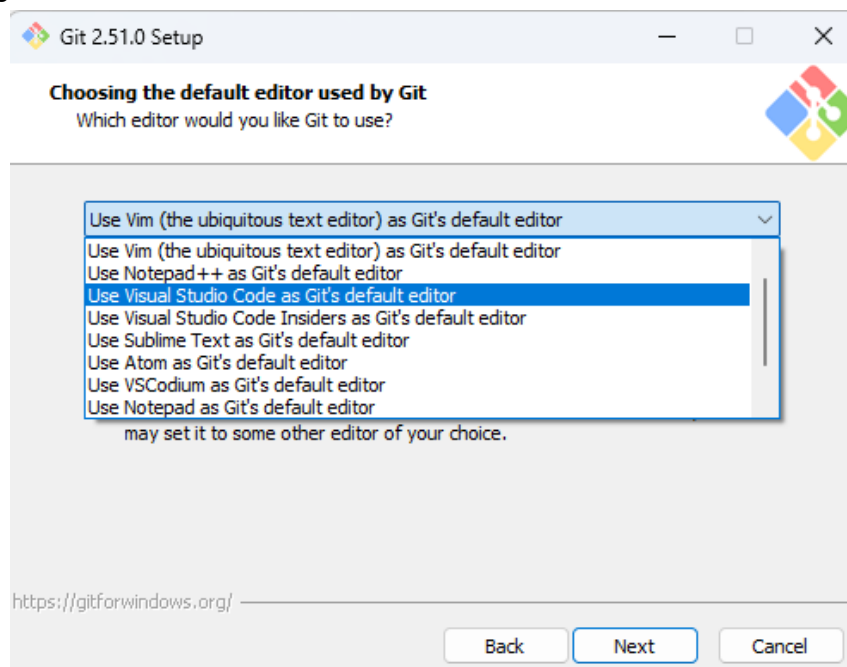
Proyecto Gestor de Cursos con Django

Instalación de git

Descargar git para tu sistema operativo



Ejecuta e instala git configurando que Visual Studio Code sea el editor predeterminado para trabajar con git



Prueba en el CMD si se instaló correctamente git.

```
C:\Users\User>git --version
git version 2.51.0.windows.1
```

Seguido de comprobar que git se haya instalado correctamente procederemos a su configuración global para todos los proyectos que se trabajen.

```
C:\Users\User>git --version
git version 2.51.0.windows.1

C:\Users\User>git config --global user.name "Carlos Aguirre"

C:\Users\User>git config --global user.email "tucorreo@correo"

C:\Users\User>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Carlos Aguirre
user.email=tucorreo@correo
```

Crear un repositorio en tu github

<https://github.com/new>

Configuraciones para crear tu primer proyecto

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1

General

Owner *

CarlosLaboratorio

Repository name *

PythonProyecto1

PythonProyecto1 is available.

Great repository names are short and memorable. How about curly-meme?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Public

Choose who can see and commit to this repository

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Python

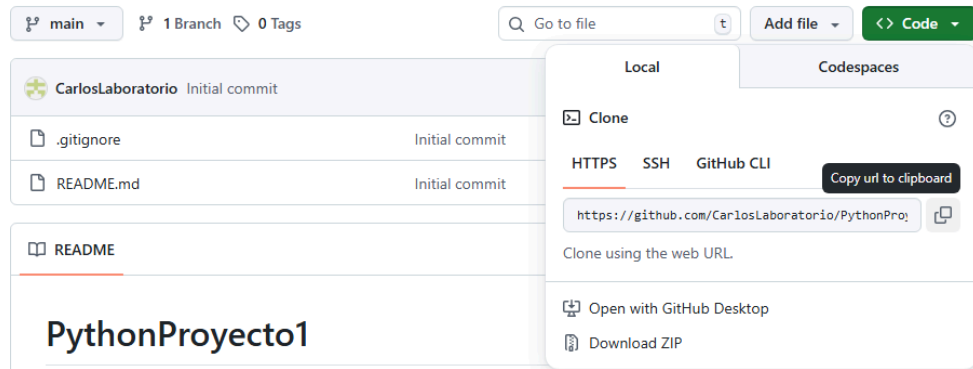
Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Copiamos el link para generar nuestra carpeta local de nuestro repositorio en el cmd de windows. Posicionarse donde se quiere guardar los trabajos.



Ir al CMD y escribe **git clone** [link-copiado](https://github.com/CarlosLaboratorio/PythonProyecto1.git)

```
C:\Users\User>git clone https://github.com/CarlosLaboratorio/PythonProyecto1.git
Cloning into 'PythonProyecto1'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
```

Posicionarse dentro de la carpeta donde descargó el proyecto y escribir

```
C:\Users\User>cd PythonProyecto1
C:\Users\User\PythonProyecto1>code .
```

Debe abrir esta carpeta con Visual Studio Code

Presionamos Ctrl + J y comprobamos que haya un proyecto git

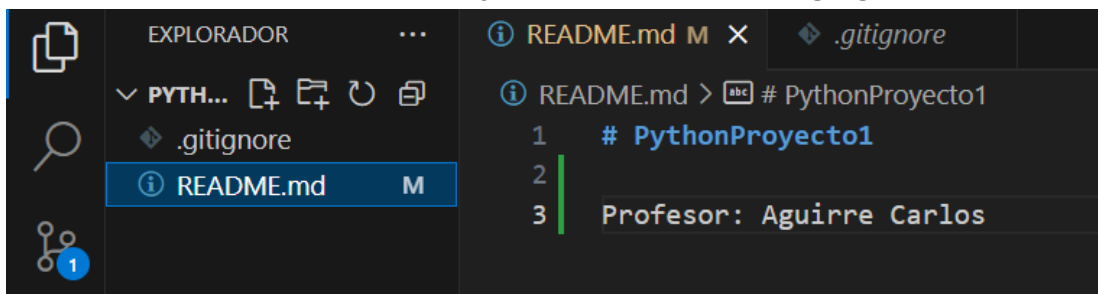
```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

PS C:\Users\User\PythonProyecto1> git status
On branch main
Your branch is up to date with 'origin/main'.

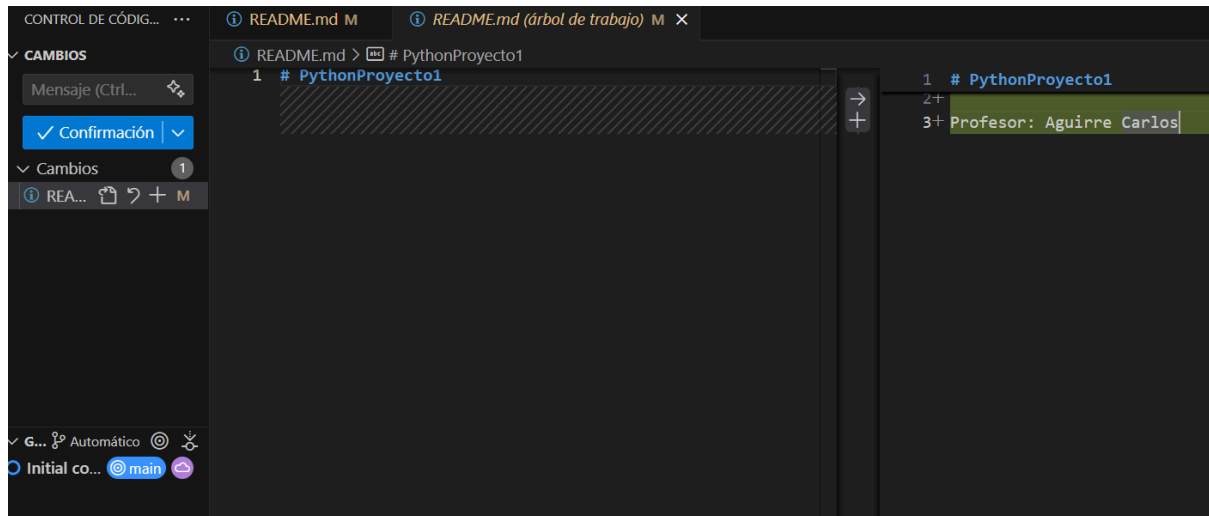
nothing to commit, working tree clean
PS C:\Users\User\PythonProyecto1>
```

Rama main Principal

Al hacer un pequeño cambio por ejemplo en [readme.md](#) agregamos nombre



Se puede ver que hay cambios por hacer, si presionas en Extensión donde vemos el número 1, podremos visualizar como estaba Antes y como estaría Después.



Para aplicar los cambios y subirlos al repositorio se utilizan los siguientes comandos:

```
PS C:\Users\User\PythonProyecto1> git add .\README.md
PS C:\Users\User\PythonProyecto1> git commit -m "Add profesor"
[main 594a10a] Add profesor
1 file changed, 3 insertions(+), 1 deletion(-)
```

Para realizar una comprobación, observa los commit existentes con el siguiente comando para una sola línea

```
PS C:\Users\User\PythonProyecto1> git log --oneline
594a10a (HEAD -> main) Add profesor
94f5647 (origin/main, origin/HEAD) Initial commit
```

Ahora sincronizamos con el repositorio de github escribiendo git push

```
PS C:\Users\User\PythonProyecto1> git push
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 316 bytes | 316.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CarlosLaboratorio/PythonProyecto1.git
94f5647..594a10a main -> main
```

Comprobamos nuevamente los commit y ya podemos ver que se encuentran en los origin

```
PS C:\Users\User\PythonProyecto1> git log --oneline
594a10a (HEAD -> main, origin/main, origin/HEAD) Add profesor
94f5647 Initial commit
```

Creando un Entorno Virtual

En primer lugar, comprobamos lo que tienes instalado de manera local con pip list. Tener todo actualizado python, pip y django.

```
PS C:\Users\User\PythonProyecto1> pip list
Package            Version
-----
asgiref            3.9.1
Django             5.2.5
Mercado-Music      0.1.0
MIDIUtil           1.2.1
pip                25.2
proyecto-clases    0.1.0
setuptools         65.5.0
sqlparse           0.5.3
tzdata             2025.2
```

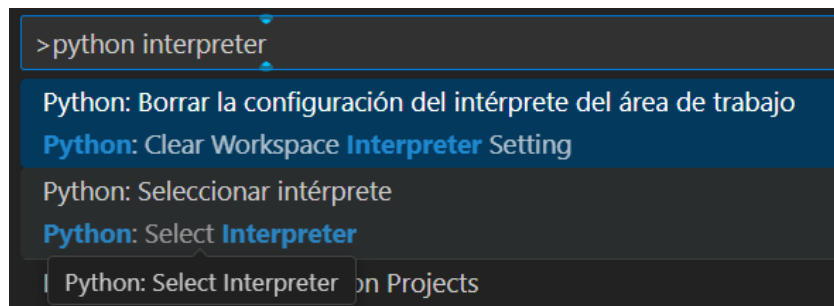
Ahora vamos a crear el entorno virtual con:
`python -m venv .venv`

```
PythonProyecto1> python -m venv .venv
```

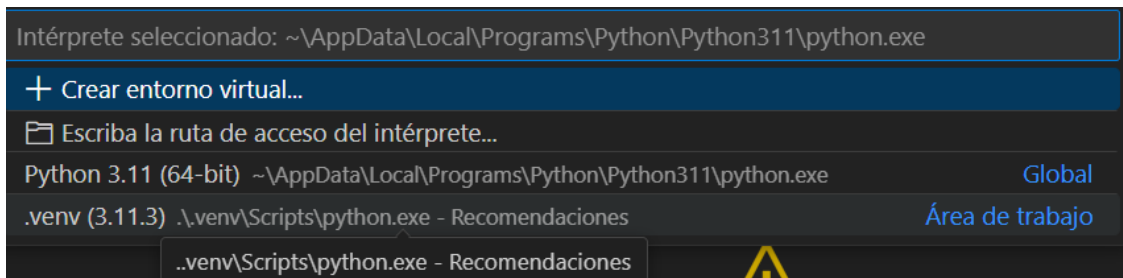
Ahora debemos indicar a Visual Studio Code que en nuestro entorno virtual no vamos a usar python del entorno local sino del entorno virtual

Presionamos Ctrl + Shift + P o F1 en el buscador

Y buscamos Python Interpreter para elegir el python venv recomendado.



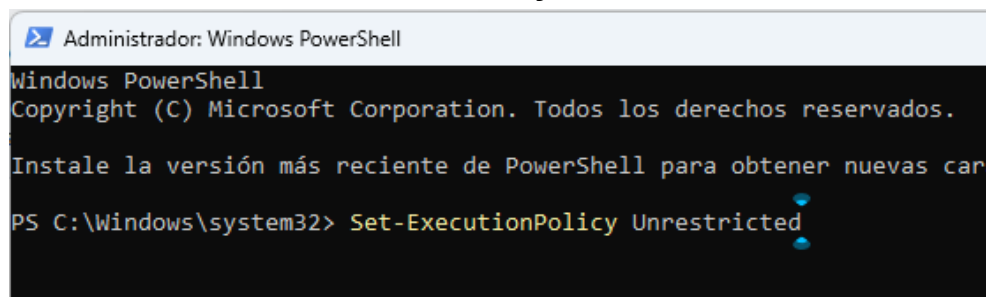
Hacemos clic en el python de Área de trabajo y se activa el venv



También podremos activar el entorno virtual ejecutando `activate`

```
(.venv) PS C:\Users\User\PythonProyecto1> .\.venv\Scripts\activate
(.venv) PS C:\Users\User\PythonProyecto1> 
```

Si Windows les da un error rojo que advierte de Scripts, ingresa en modo administrador al PowerShell y ejecuta el siguiente comando y decimos si (s) **Set-ExecutionPolicy Unrestricted**



Ahora limpiamos y cerramos la terminal, y volvemos a abrir con Ctrl + J

```
(.venv) PS C:\Users\User\PythonProyecto1> .\.venv\Scripts\activate
(.venv) PS C:\Users\User\PythonProyecto1> 
```

Una vez dentro del entorno virtual, vemos que aplicaciones tenemos con `pip list`. Solo si nos falta django lo instalamos con `pip install django`.

```
(.venv) PS C:\Users\User\PythonProyecto1> pip list
Package      Version
-----
pip          22.3.1
setuptools   65.5.0

[notice] A new release of pip available: 22.3.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\User\PythonProyecto1> pip install django
Collecting django
  Downloading django-5.2.5-py3-none-any.whl (8.3 MB)
    1.1/8.3 MB 69.0 kB/s eta 0:01:45
```

Una vez terminado, podemos ver la instrucción `django-admin` Ejecutamos en la terminal y podremos ver todos los argumentos que tenemos. Y podemos especificar alguna ayuda para algún subcomando, por ejemplo:

django-admin help startproject

Ahora sí, empezamos a crear el proyecto en Django!

Ejemplo: `django-admin startproject Proyecto1` .

```
(.venv) PS C:\Users\User\PythonProyecto1> django-admin startproject Proyecto1
```

Nos dirigimos al directorio de Proyecto1 y ejecutamos
python [manage.py](#) runserver
Probamos en el navegador <http://127.0.0.1:8000/>



The install worked successfully! Congratulations!

View [release notes](#) for Django 5.2

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

django

Creación de una App en Proyecto1

```
o1\Proyecto1> python manage.py startapp myapp
```

Tendrás una carpeta exclusiva para esta app.

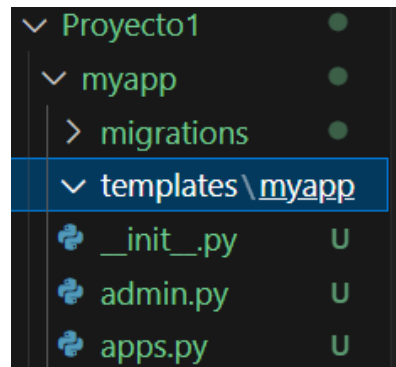
Las aplicaciones se utilizan para obtener funcionalidades específicas para tu proyecto, y lo puedas utilizar en todo el proyecto sin necesidad de repetir código, como la idea de una función en programación.

Luego, modifica para agregar la app en el archivo settings.py

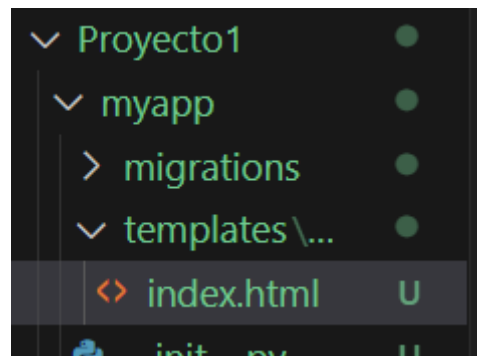
```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'myapp',  
]
```

Creación de un Template

Genera la carpeta templates\myapp en myapp



Crea index.html en dicha carpeta:



Escribe el siguiente código en index.html

```
Proyecto1 > myapp > templates > myapp > <> index.html > html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Mi Django App</title>
7  </head>
8  <body>
9      <h1>Hola Django!</h1>
10     <p> {{ mensaje }}</p>
11 </body>
12 </html>
```

El uso de {{ }} no es exclusivo de html sino es un llamado a django para mostrar un mensaje que se especifica en [views.py](#) de la siguiente manera:

```
myapp > views.py > index
1  from django.shortcuts import render
2
3  # Create your views here.
4  def index(request):
5      context = {"mensaje": "Bienvenidos a mi aplicación Django"}
6      return render(request, "myapp/index.html", context)
```

Modificación en [urls.py](#) para abrir el index.html en la ruta prueba/

```
from django.contrib import admin
from django.urls import path

from myapp import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path("prueba/", views.index, name="index")
]
```

Modificación en [urls.py](#) para abrir el index.html en la ruta raíz:

```
16
17 from django.contrib import admin
18 from django.urls import path
19
20 from myapp import views
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path("", views.index, name="index")
25 ]
26
```

En el archivo models.py

```
from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return f"{self.nombre} {self.apellido}"

class Profesor(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()
    profesion = models.CharField(max_length=100)

    def __str__(self):
        return f"{self.nombre} {self.apellido} - {self.profesion}"

class Curso(models.Model):
    nombre = models.CharField(max_length=100)
```

```
camada = models.IntegerField()

def __str__(self):
    return self.nombre

class Entregable(models.Model):
    nombre = models.CharField(max_length=100)
    fecha_entrega = models.DateField()
    entregado = models.BooleanField()

    def __str__(self):
        return self.nombre
```

Migración de Modelos

Para crear las tablas en la base de datos, ejecutamos las migraciones:

```
python manage.py makemigrations
python manage.py migrate
```

[Copiar](#)

Vista (View)

La vista maneja la lógica de negocio y la interacción del usuario. Se encarga de procesar las solicitudes y devolver respuestas. En myApp, creamos vistas para manejar las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) para nuestros modelos.

En el archivo views.py de myapp, definimos nuestras vistas:

```
from django.shortcuts import render, get_object_or_404
from .models import Estudiante, Profesor, Curso, Entregable

def lista_estudiantes(request):
    estudiantes = Estudiante.objects.all()
    return render(request, 'myApp/estudiantes_list.html',
{'estudiantes': estudiantes})

def detalle_estudiante(request, pk):
    estudiante = get_object_or_404(Estudiante, pk=pk)
    return render(request, 'myApp/estudiante_detail.html',
{'estudiante': estudiante})
```

Estas vistas obtienen datos de los modelos y los pasan a las plantillas para su presentación.

Plantilla (Template)

Las plantillas definen cómo se presentan los datos al usuario. Utilizan el lenguaje de plantillas de Django para generar HTML dinámico.

Plantilla `estudiantes_list.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>Lista de Estudiantes</title>
</head>
<body>
  <h1>Lista de Estudiantes</h1>
  <ul>
    {% for estudiante in estudiantes %}
      <li>{{ estudiante.nombre }} {{ estudiante.apellido }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

Plantilla `estudiante_detail.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>Detalle del Estudiante</title>
</head>
<body>
  <h1>{{ estudiante.nombre }} {{ estudiante.apellido }}</h1>
  <p>Email: {{ estudiante.email }}</p>
</body>
</html>
```

Configuración de URLs

Finalmente, configuramos las URLs para que las vistas sean accesibles. Editamos el archivo `urls.py` de `myapp` y el archivo principal de URLs del proyecto.

`myapp/urls.py`:

```
from django.urls import path
from .views import lista_estudiantes, detalle_estudiante

urlpatterns = [
    path('estudiantes/', lista_estudiantes, name='lista_estudiantes'),
    path('estudiantes/<int:pk>/', detalle_estudiante,
name='detalle_estudiante'),
]
```

Proyecto1/urls.py:

```
from django.contrib import admin
from django.urls import include, path

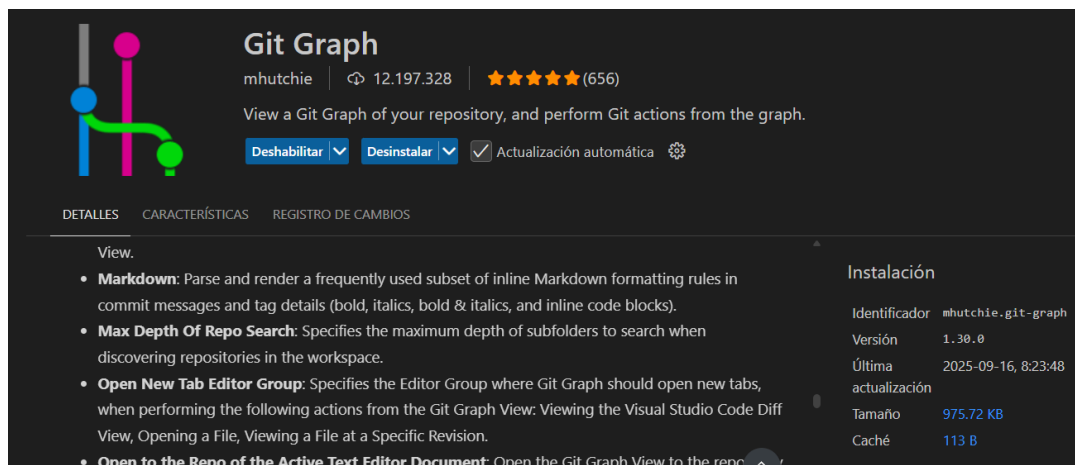
urlpatterns = [
    path('admin/', admin.site.urls),
    path('myApp/', include('myApp.urls')),
]
```

Conclusión

El patrón MTV en Django ayuda a organizar el código de manera eficiente. Hemos creado un proyecto y una aplicación myapp que incluye modelos, vistas y plantillas para manejar estudiantes, profesores, entregables y cursos. Con esta estructura, es más fácil gestionar y mantener la aplicación a medida que crece en complejidad.

Control de Versiones

Instalación de Extensiones extras:



Git Graph
mhutchie | 12.197.328 | ★★★★★ (656)

View a Git Graph of your repository, and perform Git actions from the graph.

[Deshabilitar](#) [Desinstalar](#) ☒ Actualización automática ⚙️

DETALLES | **CARACTERÍSTICAS** | REGISTRO DE CAMBIOS

View.

- **Markdown:** Parse and render a frequently used subset of inline Markdown formatting rules in commit messages and tag details (bold, italics, bold & italics, and inline code blocks).
- **Max Depth Of Repo Search:** Specifies the maximum depth of subfolders to search when discovering repositories in the workspace.
- **Open New Tab Editor Group:** Specifies the Editor Group where Git Graph should open new tabs, when performing the following actions from the Git Graph View: Viewing the Visual Studio Code Diff View, Opening a File, Viewing a File at a Specific Revision.
- **Open to the Repo of the Active Text Editor Document:** Open the Git Graph View to the repo

Instalación

Identificador	mhutchie.git-graph
Versión	1.30.0
Última actualización	2025-09-16, 8:23:48
Tamaño	975.72 KB
Caché	113 B

The screenshot displays two extension pages from the Visual Studio Code Marketplace. The top extension is 'Django Support' by Al Mahdi, which has 40,840 downloads and a 5-star rating. It is described as 'The ultimate django framework support' and includes an 'Instalar' button and a checkbox for 'Actualización automática'. Below the extension name, there are tabs for 'DETALLES', 'CARACTERÍSTICAS', and 'REGISTRO DE CAMBIOS'. The main content area shows 'Django support for Visual Studio Code' with a 'Zero config' note and a code snippet for an HTML template. To the right, a 'Marketplace' sidebar shows the extension's identifier, version (1.0.0), publication date (2022-04-03), and last update (2024-10-16). The bottom extension is 'SQLite Viewer' by Florian Klampfer, with 2,300,676 downloads and a 5-star rating. It is described as 'SQLite Viewer for VS Code' and also features an 'Instalar' button and an 'Actualización automática' checkbox. Similar to the first extension, it has tabs for details, characteristics, and change log. The main content area shows 'SQLite Viewer for VS Code' with a list of supported languages (Deutsch, Français, Português, 日本語, Español, 한국어) and a brief description. The 'Marketplace' sidebar on the right shows its identifier, version (0.10.6), publication date (2021-10-01), and last update (2025-08-09).

Creación de Modelos en Django

¿Qué es un Modelo en Django?

Un modelo en Django define los campos y comportamientos de los datos que deseas almacenar. Los modelos proporcionan una API de alto nivel para crear, leer, actualizar y eliminar registros en la base de datos sin necesidad de escribir SQL directamente.

Importancia de los Modelos en Django:

- **Abstracción de la Base de Datos:** Permiten trabajar con datos sin preocuparse por los detalles específicos del SQL.
- **Consistencia:** Facilitan la validación y la consistencia de los datos.
- **Flexibilidad:** Proporcionan una manera sencilla de migrar cambios en la estructura de datos a lo largo del desarrollo del proyecto.

Proceso para Crear un Modelo en Django

A continuación, se detalla el proceso para crear un modelo en la aplicación AppCoder de un proyecto Django.

Paso 1: Definir el Modelo

Para definir un modelo, necesitas editar el archivo `models.py` de tu aplicación. Vamos a crear modelos para Estudiante, Profesor, Curso y Entregable en `myapp`.

`myapp/models.py`:

Paso 2: Realizar Migraciones

Django utiliza un sistema de migraciones para aplicar los cambios en la estructura de la base de datos. Después de definir los modelos, necesitas crear y aplicar las migraciones.

Crear las migraciones: Genera los archivos de migración que describen los cambios en los modelos.

`python manage.py makemigrations`

Aplicar las migraciones: Aplica los cambios en la base de datos.

`python manage.py migrate`

Paso 3: Registrar los Modelos en el Administrador de Django

Para poder gestionar los modelos desde el administrador de Django, debes registrarlos en el archivo `admin.py` de tu aplicación.

`myapp/admin.py`:

```
from django.contrib import admin
from .models import Estudiante, Profesor, Curso, Entregable

admin.site.register(Estudiante)
admin.site.register(Profesor)
admin.site.register(Curso)
admin.site.register(Entregable)
```

Paso 4: Crear y Probar el Superusuario

Para acceder al panel de administración, necesitas crear un superusuario (administrador).

Crear superusuario:

python manage.py createsuperuser

Iniciar el servidor de desarrollo:

python manage.py runserver

Acceder al panel de administración: Visita <http://127.0.0.1:8000/admin/> en tu navegador e inicia sesión con las credenciales del superusuario que creaste.

Ejemplo Completo

A continuación se muestra un ejemplo completo del proceso para crear y gestionar modelos en la aplicación myapp.

Definir modelos en models.py:

```
from django.db import models

class Estudiante(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return f"{self.nombre} {self.apellido}"

class Profesor(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()
    profesion = models.CharField(max_length=100)

    def __str__(self):
        return f"{self.nombre} {self.apellido} - {self.profesion}"

class Curso(models.Model):
    nombre = models.CharField(max_length=100)
    camada = models.IntegerField()

    def __str__(self):
        return self.nombre

class Entregable(models.Model):
    nombre = models.CharField(max_length=100)
```

```
fecha_entrega = models.DateField()
entregado = models.BooleanField()

def __str__(self):
    return self.nombre
```

Crear y aplicar migraciones:

python manage.py makemigrations

python manage.py migrate

Registrar modelos en admin.py:

```
from django.contrib import admin
from .models import Estudiante, Profesor, Curso, Entregable

admin.site.register(Estudiante)
admin.site.register(Profesor)
admin.site.register(Curso)
admin.site.register(Entregable)
```

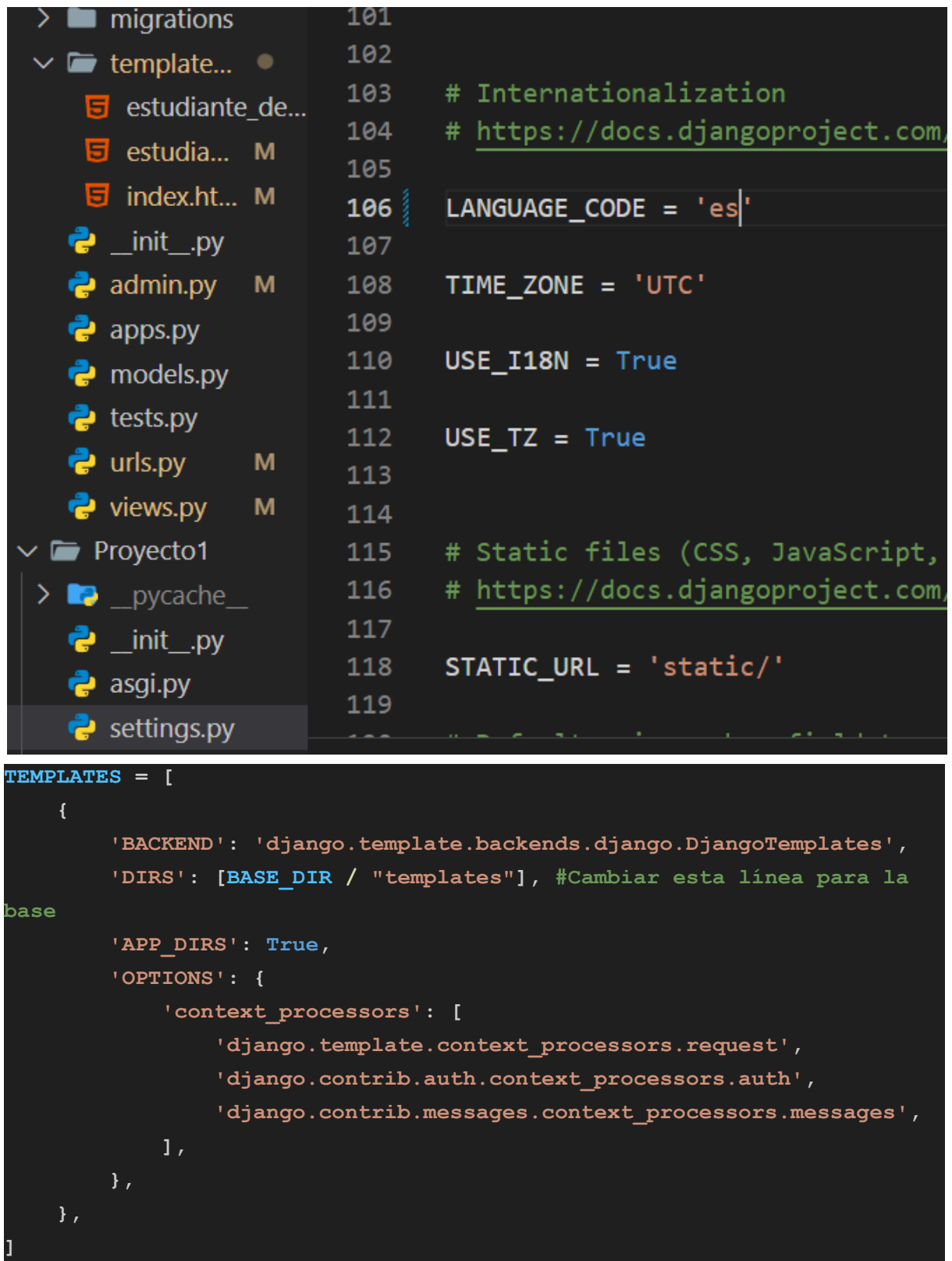
Crear superusuario y ejecutar el servidor: bash

python manage.py createsuperuser

python manage.py runserver

Accede al panel de administración de Django para gestionar los datos de Estudiante, Profesor, Curso y Entregable.

Cambia el idioma y TEMPLATES en [settings.py](#) del proyecto principal:



The image shows a code editor with a file explorer on the left and code in the main area. The file explorer shows a project structure with folders like 'migrations', 'template...', and 'Proyecto1'. The main area displays the 'settings.py' file with the following code:

```
101
102
103 # Internationalization
104 # https://docs.djangoproject.com/
105
106 LANGUAGE_CODE = 'es'
107
108 TIME_ZONE = 'UTC'
109
110 USE_I18N = True
111
112 USE_TZ = True
113
114
115 # Static files (CSS, JavaScript,
116 # https://docs.djangoproject.com/
117
118 STATIC_URL = 'static/'
119
```

Below this, the 'TEMPLATES' configuration is shown:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / "templates"], #Cambiar esta línea para la
base
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Herencia de Templates

La herencia de templates es una técnica en Django que permite reutilizar código HTML para mantener una estructura consistente en varias páginas web. Este método se basa en definir un template "padre" que

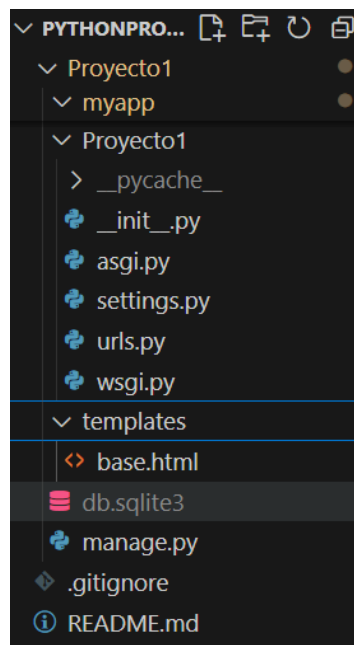
contiene elementos comunes a todas las páginas (como la barra de navegación, el pie de página, y el logo), y luego extender ese template en los templates "hijos" que heredan y pueden modificar su contenido específico.

¿Cómo definir un template "padre"?

Crear el Template Padre:

El primer paso es crear un archivo HTML que actuará como el template base. Este archivo incluirá las partes de la página web que serán comunes a todas las páginas.

Crea el archivo base.html en la carpeta global Proyecto1:



Ejemplo de base.html:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>{% block title %}My Site{% endblock %}</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.m
in.css">
  </head>
  <body>
    <header>
      <nav class="navbar navbar-light bg-light static-top">
```

```
<div class="container">
  <a class="navbar-brand" href="{% url 'myapp:index'
%}">Inicio</a>
  <a class="navbar-brand" href="{% url 'myapp:cursos '
%}">Cursos</a>
  <a class="navbar-brand" href="{% url
'myapp:profesores' %}">Profesores</a>
  <a class="navbar-brand" href="{% url
'myapp:estudiantes' %}">Estudiantes</a>
  <a class="navbar-brand" href="{% url
'myapp:entregables' %}">Entregables</a>
  <a class="btn btn-primary"
href="#NADAAUN">INICIAR</a> </div>
</nav>
</header>
<main>
  {% block content %} <!-- Contenido específico de cada
página irá aquí --> {% endblock %}
</main>
<footer> <p>&copy; 2025 My Site</p>
</footer>
</body>
</html>
```

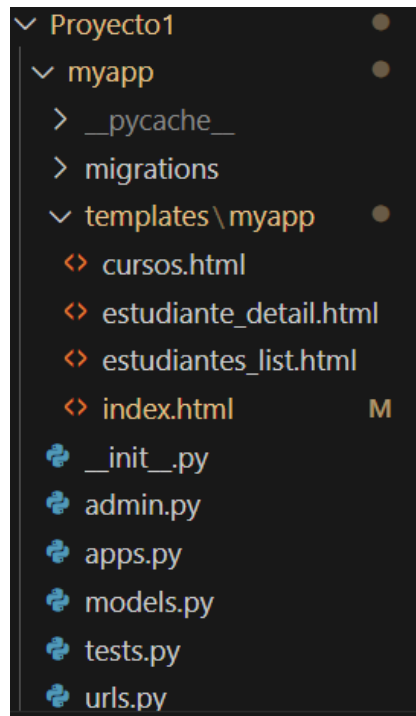
¿Cómo extender un template "padre" en un template "hijo"?

Crear el Template Hijo:

Los templates hijos extienden el template padre usando la directiva `{% extends 'ruta/al/template/base.html' %}`.

Luego, definen el contenido específico para los bloques que se desea modificar.

Crea los siguientes templates en la carpeta `templates/myapp` de la aplicación `myapp`:



Ejemplo de cursos.html:

```
{% extends 'base.html' %}
{% block title %}Página de Cursos{% endblock %}
{% block content %}
    <h1>Bienvenido a la página de cursos</h1>
    <p>Aquí encontrarás todos los cursos disponibles.</p>
{% endblock %}
```

Beneficios de la Herencia de Templates:

Consistencia: Asegura que todos los templates compartan un diseño común, lo que facilita la actualización y el mantenimiento del sitio web.

Reutilización de Código: Evita la duplicación de código, ya que los elementos comunes se definen una vez en el template padre.

Flexibilidad: Permite que cada página web tenga contenido específico mientras mantiene la estructura general del sitio.

La herencia de templates en Django es una herramienta poderosa que simplifica la gestión y el mantenimiento de sitios web, permitiendo una estructura coherente y un diseño uniforme en todas las páginas.

A partir de ahora puedes seguir agregando templates, siempre ajustando y creando las conexiones con los mismos pasos que ya se realizaron antes:

Crear las vistas (**views.py**).

Agregar las rutas (**urls.py**).

Crear los templates (**profesores.html**, **entregables.html**).

Extender de **base.html** para mantener el mismo diseño.

Navegación entre Templates:

Para asegurarse de que el menú de navegación funcione correctamente, es importante definir las vistas en **views.py** y las URLs correspondientes en el archivo **urls.py** de Django.

Ejemplo de configuración en **views.py**:

```
from django.shortcuts import render, get_object_or_404
from .models import Estudiante, Profesor, Curso, Entregable

def index(request):
    context = {"mensaje": "Bienvenidos a mi aplicación Django"}
    estudiantes = Estudiante.objects.all()
    return render(request, 'myapp/index.html', context)

def estudiantes(request):
    estudiantes = Estudiante.objects.all()
    return render(request, 'myapp/estudiantes_list.html',
{'estudiantes': estudiantes})

def cursos(request):
    cursos = Curso.objects.all()
    return render(request, 'myapp/cursos.html', {'cursos': cursos})

def profesores(request):
    profesores = Profesor.objects.all()
    return render(request, '', {'profesores': profesores})

def entregables(request):
    entregables = Entregable.objects.all()
    return render(request, '', {'entregables': entregables})
```

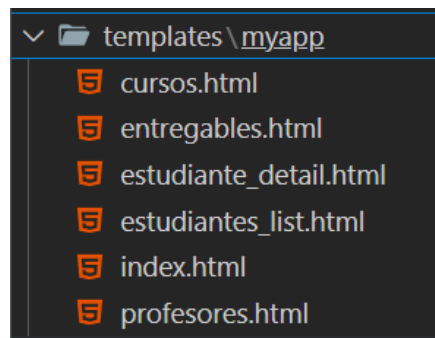
Ejemplo de configuración en urls.py:

```
from django.urls import path
from myapp import views

app_name = "myapp"
urlpatterns = [
    path("", views.index, name="index"),
    path('cursos/', views.cursos, name="cursos"),
    path('profesores/', views.profesores, name="profesores"),
    path('estudiantes/', views.estudiantes, name="estudiantes"),
    path('entregables/', views.entregables, name="entregables"),
]
```

Creación de los Templates Profesores y Entregables

Dirigirse a la carpeta templates/myapp y crear los templates profesores.html y entregables.html. Luego, completar los enlaces a dichos archivos en views.py, el archivo urls.py, ya se realizó la edición completa para todas las secciones.



Edición de views.py

```
def profesores(request):
    profesores = Profesor.objects.all()
    return render(request, 'myapp/profesores.html', {'profesores':
profesores})

def entregables(request):
    entregables = Entregable.objects.all()
    return render(request, 'myapp/entregables.html', {'entregables':
entregables})
```

Edición de los Templates

En esta ocasión, cómo ya instalamos bootstrap en base.html, vamos a empezar a experimentar con las diferentes presentaciones que nos brinda esta herramienta creada por Twitter.

En estudiantes.html, vemos cómo muestra la información de la base de datos con un formato predeterminado de las listas en html.

En el html de profesores, vamos a presentar los datos a través de tarjetas con diferentes estilos usando grid de bootstrap como veremos a continuación.

Baez Dario

Email: que-mosca@hola.com

Profesión: Profe y licenciado

El código completo para extender el html con base.html y la inclusión del bootstrap específico para este formato es:

```
{% extends 'base.html' %}
{% block title %} Lista de Profesores {% endblock %}
{% block content %}
<div class="container mt-4">
  <h1 class="text-center mb-4">Lista de Profesores</h1>
  <div class="row">
    {% for profesor in profesores %}
    <div class="col-md-4 col-sm-6 mb-4">
      <div class="card shadow-sm h-100">
        <div class="card-body">
          <h5 class="card-title text-primary">
            {{ profesor.apellido }} {{ profesor.nombre }}
          </h5>
          <p class="card-text mb-1">
            <strong>Email:</strong> {{ profesor.email }}
          </p>
          <p class="card-text">
            <strong>Profesión:</strong> {{
profesor.profesion }}
          </p>
        </div>
      </div>
    </div>
    {% endfor %}
  </div>
</div>
```

```

    </div>
    {% empty %}
    <p class="text-muted text-center">No hay profesores
registrados.</p>
    {% endfor %}
  </div>
</div>
{% endblock %}

```

Como observarán, siempre se repite una cierta estructura, y los cambios se ven dentro del bloque de title y dentro del bloque de content.

```

{% extends 'base.html' %}
{% block title %}      {% endblock %}
{% block content %}

{% endblock %}

```

En el html de entregables, vamos a presentar los datos a través de una tabla con diferentes estilos tanto en el encabezado como en el resto de la estructura.

Trabajos Entregados

Nombre	Fecha de Entrega	Entregado
Página en HTML	21 de septiembre de 2025	Sí

El código que debemos agregar para obtener esta tabla es el siguiente:

```

<div class="container mt-4">
  <h1 class="text-center mb-4">Trabajos Entregados</h1>

  <div class="table-responsive">
    <table class="table table-striped table-hover align-middle
shadow-sm">
      <thead class="table-dark">
        <tr>
          <th scope="col">Nombre</th>
          <th scope="col">Fecha de Entrega</th>
          <th scope="col">Entregado</th>
        </tr>
      </thead>

```

```
<tbody>
  {% for entregable in entregables %}
    <tr>
      <td>{{ entregable.nombre }}</td>
      <td>{{ entregable.fecha_entrega }}</td>
      <td>
        {% if entregable.entregado %}
          <span class="badge
bg-success">Sí</span>
        {% else %}
          <span class="badge bg-danger">No</span>
        {% endif %}
      </td>
    </tr>
  {% empty %}
    <tr>
      <td colspan="3" class="text-center text-muted">
        No hay entregables registrados.
      </td>
    </tr>
  {% endfor %}
</tbody>
</table>
</div>
</div>
```

Crea un [readme.md](#) impactante en la página

[GitHub Profile Readme Generator](#)

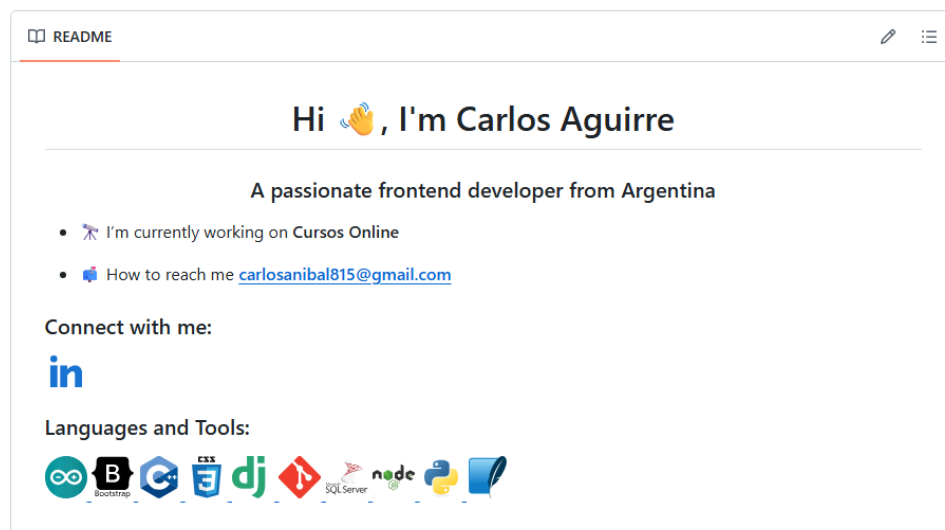
```
<h1 align="center">Hi 🙋, I'm Carlos Aguirre</h1>
<h3 align="center">A passionate frontend developer from Argentina</h3>

- ✈️ I'm currently working on **Cursos Online**

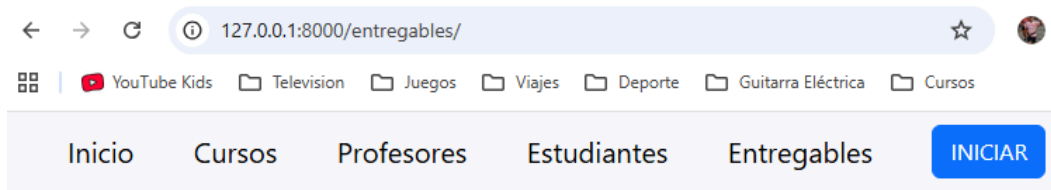
- 📧 How to reach me **carlosanibal815@gmail.com**

<h3 align="left">Connect with me:</h3>
<p align="left">
<a href="https://linkedin.com/in/carlos-anibal-aguirre-b768a0a9/" target="
generator/master/src/images/icons/Social/linkedin-alt.svg" alt="carlos-an
</p>
```

Copia y pega el código generado en tu archivo [readme.md](#), luego mira su diseño en Github.



Visualización momentánea del proyecto



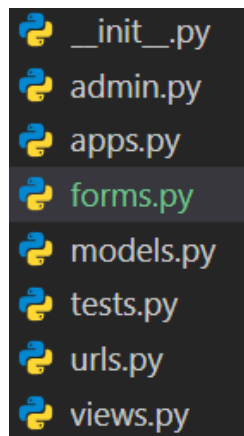
Trabajos Entregados

Nombre	Fecha de Entrega	Entregado
Página en HTML	21 de septiembre de 2025	Sí

© 2025 My Site

Creación de Formularios en Django

Crea el archivo [forms.py](#) en la app.



Ejemplo de un formulario básico para agregar un curso ([forms.py](#)):

```
from django import forms

class CursoFormulario(forms.Form):
    nombre = forms.CharField(max_length=100)
    camada = forms.IntegerField()
```

En el archivo views.py, se define una vista que manejará el formulario.

```
from django.shortcuts import render
from .forms import CursoFormulario

def cursoFormulario(request):
    if request.method == 'POST':
        form = CursoFormulario(request.POST)
        if form.is_valid(): # Procesar los datos del formulario
            nombre = form.cleaned_data['nombre']
```

```

        camada = form.cleaned_data['camada'] # Guardar los datos en
la base de datos
        curso = Curso(nombre=nombre, camada=camada)
        curso.save()

        return render(request, 'myapp/curso_exito.html')

    else:
        form = CursoFormulario()

    return render(request, 'myapp/curso_formulario.html', {'form':
form})

```

Es necesario registrar la vista en el archivo urls.py para que sea accesible desde una URL.

```

from django.urls import path
from . import views

app_name = "myapp"

urlpatterns = [
    path("", views.index, name="index"),
    path('cursos/', views.cursos, name="cursos"),
    path('profesores/', views.profesores, name="profesores"),
    path('estudiantes/', views.estudiantes, name="estudiantes"),
    path('entregables/', views.entregables, name="entregables"),
    path('cursoFormulario/', views.cursoFormulario,
name='cursoFormulario'),
]

```

Se crea una plantilla HTML curso_formulario.html que renderizará el formulario. Luego, extenderlo con base.html para mantener el menú de navegación presente.

```

<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <title>Formulario - Agregar Curso</title>
    </head>
    <body>
        <h1>Agregar Curso</h1>
        <form action="/cursoFormulario/" method="post">
            {% csrf_token %} {{ form.as_p }}
            <button type="submit">Enviar</button>
        </form>
    </body>
</html>

```

Llegó tu turno de utilizar todo lo aprendido e ir modificando todo por un diseño moderno y responsivo que se adapte a todas las pantallas. Cada vez que tu práctica llegue a la estructura básica para tu proyecto, busca el bootstrap ideal para tu proyecto en [Bootstrap](#). También puedes llevar tu código a tu inteligencia artificial de preferencia y pedir que te elabore un diseño moderno y responsivo del código html a actualizar.

En cursos.html podrás elegir cómo ir hacia este formulario, como por ejemplo con un botón secundario de bootstrap. Además, ya le daremos un buen diseño a todo el html de cursos.

```
{% extends 'base.html' %}
{% block title %}Página de Cursos{% endblock %}
{% block content %}
<div class="container mt-4">
    <h1 class="text-center mb-4">Bienvenido a la página de Cursos</h1>
    <div class="text-end mb-4"> <a class="btn btn-secondary" href="{%
url 'myapp:cursoFormulario' %}">Agregar Curso</a> </div>
    <div class="row">
        {% for curso in cursos %}
        <div class="col-md-4 col-sm-6 mb-4">
            <div class="card shadow-sm h-100">
                <div class="card-body">
                    <h5 class="card-title text-primary">
                        {{ curso.nombre }}
                    </h5>
                    <p class="card-text mb-1">
                        <strong>Camada:</strong> {{ curso.camada }}
                    </p>
                    <p class="card-text">
                        <strong>Detalle:</strong> {{ curso.camada }}
                    </p>
                </div>
            </div>
        </div>
        {% empty %}
        <p class="text-muted text-center">No hay cursos
disponibles.</p>
        {% endfor %}
    </div>
</div>
```

```
</div>  
{% endblock %}
```

A partir de ahora, con todas estas prácticas y conocimientos adquiridos, ya puedes hacer todo el sitio amigable, en donde, siempre se mostrará el menú de navegación y el base.html completo en todas las páginas. Puedes mejorar las vistas de todos los html, elegir presentar datos con listas, grilla de tarjetas o tablas. Añadir botones y formularios para todos: cursos, profesores, estudiantes y entregables, y un formulario para iniciar sesión inicial. Puedes elaborar todas las vistas y funcionalidades mientras seguiremos avanzando con el funcionamiento de los Métodos GET y POST. Ya estamos muy cerca a la introducción a CRUD (Crear, Leer, Actualizar y Borrar).

Paralelamente a estas prácticas, ya inicia tu proyecto de la carrera con el grupo que estás trabajando. Deben estar decididos al modelo de base de datos ejemplo que trabajan, y ya crear todo el proyecto en Django según lo que trabajamos. Luego, a medida que necesiten ajustes, van editando su proyecto según necesidades, solicitudes de profesores de la carrera y propuestas de mejoras que tengan por los demás medios. Vamos que falta poco para finalizar con la Tecnicatura en Desarrollo de Software.

Aquí tienen más ejemplos de otros proyectos Django:

📁 **Proyectos Django**

Formulario - Agregar Curso

Vamos a mejorar el formulario a un estilo moderno y responsivo instalando una nueva herramienta muy utilizada en django.

Usaremos `{{ form.campo|add_class:"form-control" }}` → para que cada campo se vea con estilo Bootstrap.

Para esto instalamos django-widget-tweaks:

```
pip install django-widget-tweaks
```

Y en settings.py agregar 'widget_tweaks', en INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
    'widget_tweaks',
]
```

Luego en el template {% load widget_tweaks %}:

```
{% extends 'base.html' %}
{% load widget_tweaks %}
{% block title %} Formulario - Agregar Curso {% endblock %}
{% block content %}
<div class="container mt-4">
    <h1 class="text-center mb-4">Agregar Curso</h1>
```

Y por último, vemos el código del template dentro del block content:

```
{% block content %}
<div class="container mt-4">
    <h1 class="text-center mb-4">Agregar Curso</h1>

    <div class="row justify-content-center">
        <div class="col-md-6 col-lg-5">
            <div class="card shadow-sm">
                <div class="card-body">
                    <form action="{% url 'myapp:cursoFormulario' %}"
method="post" novalidate>
                        {% csrf_token %}

                        <!-- Renderizar campos manualmente para aplicar
Bootstrap -->

                        <div class="mb-3">
                            <label for="id_nombre"
class="form-label">Nombre del Curso</label>
                            {{ form.nombre|add_class:"form-control" }}
                        </div>

                        <div class="mb-3">
```

```

        <label for="id_camada"
class="form-label">Camada</label>
        {{ form.camada|add_class:"form-control" }}
    </div>

    <div class="d-grid">
        <button type="submit" class="btn
btn-primary">
            <i class="bi bi-check-circle"></i>
Enviar
        </button>
    </div>
</form>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

Template - Curso agregado

Ya está todo el código para crear el curso, ya se agregan a tu base de datos. Aunque, hacerlo, te dará error porque no existe tu html que confirme el curso. En views.py ya incorporamos código para que muestre una vista al crear el curso.

En templates/myapp crear curso_exito.html con este código básico:

```

{% extends 'base.html' %}
{% block content %}
<div class="container text-center mt-4">
    <h2>✅ Curso agregado con éxito</h2>
    <a href="{% url 'myapp: cursos' %}" class="btn btn-primary
mt-3">Volver a la lista de cursos</a>
</div>
{% endblock %}

```

Luego, ya puedes probar y volver a la lista de cursos para notar los cursos nuevos existentes.

Bienvenido a la página de Cursos

[Agregar Curso](#)

Python

Camada: 160

Detalle: 160

Desarrollo Web

Camada: 125

Detalle: 125

Inteligencia Artificial

Camada: 180

Detalle: 180

Ciencia de datos

Camada: 126

Detalle: 126

HTML y CSS

Camada: 80

Detalle: 80

Robotica

Camada: 220

Detalle: 220

Arduino Profesional

Camada: 75

Detalle: 75

CRUD completo para Profesores

Ya realizamos la creación de cursos y nuestra base de datos se actualiza normalmente. Vamos a hacer todo el CRUD completo para Profesores, así ya tendrán la parte práctica para aplicar CRUD a todas las tablas por ustedes mismos.

Formulario - Agregar Profesor

Ya existe el modelo para profesor y esta perfecto en [models.py](#)

```
class Profesor(models.Model):
    nombre = models.CharField(max_length=100)
    apellido = models.CharField(max_length=100)
    email = models.EmailField()
    profesion = models.CharField(max_length=100)

    def __str__(self):
        return f"{self.nombre} {self.apellido} - {self.profesion}"
```

Ahora debes agregar el form para Profesores debajo del form de Cursos en forms.py como se ve a continuación:

```
from django import forms

class CursoFormulario(forms.Form):
    nombre = forms.CharField(max_length=100)
    camada = forms.IntegerField()

class ProfesorFormulario(forms.Form):
    nombre = forms.CharField(max_length=100, label="Nombre")
    apellido = forms.CharField(max_length=100, label="Apellido")
    email = forms.EmailField(label="Correo Electrónico")
    profesion = forms.CharField(max_length=100, label="Profesión")
```

En este formulario, puedes notar que se aplican validaciones automáticas como la de verificar si el email es válido.

En views.py hacemos pequeñas modificaciones en las importaciones y luego, al final, agregamos la función profesorFormulario como la de Cursos debajo de cursoFormulario:

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Estudiante, Profesor, Curso, Entregable
from .forms import CursoFormulario, ProfesorFormulario

def profesorFormulario(request):
    if request.method == 'POST':
        form = ProfesorFormulario(request.POST)
        if form.is_valid():
            nombre = form.cleaned_data['nombre']
            apellido = form.cleaned_data['apellido']
            email = form.cleaned_data['email']
            profesion = form.cleaned_data['profesion']
            profesor = Profesor(nombre=nombre, apellido=apellido,
email=email, profesion=profesion)
            profesor.save()
            return redirect('myapp:profesores') # Redirige a la lista
de profesores
        else:
            form = ProfesorFormulario()
            return render(request, 'myapp/profesor_formulario.html', {'form':
form})
```

En urls.py de la app myapp agregar la línea de profesorFormulario:

```
from django.urls import path
from . import views

app_name = "myapp"
urlpatterns = [
    path("", views.index, name="index"),
    path('cursos/', views.cursos, name="cursos"),
    path('profesores/', views.profesores, name="profesores"),
    path('estudiantes/', views.estudiantes, name="estudiantes"),
    path('entregables/', views.entregables, name="entregables"),
    path('cursoFormulario/', views.cursoFormulario,
name='cursoFormulario'),
```

```
path('profesorFormulario/', views.profesorFormulario,
name='profesorFormulario'),
]
```

En templates/myapp crear profesor_formulario.html con este código (usamos Bootstrap + widget_tweaks como el de cursos):

```
{% extends 'base.html' %}
{% load widget_tweaks %}

{% block title %}Agregar Profesor{% endblock %}

{% block content %}
<div class="container mt-4">
  <h1 class="text-center mb-4">Agregar Profesor</h1>

  <div class="row justify-content-center">
    <div class="col-md-6 col-lg-5">
      <div class="card shadow-sm">
        <div class="card-body">
          <form action="{% url 'myapp:profesorFormulario' %}"
method="post" novalidate>
            {% csrf_token %}

            <!-- Mostrar errores generales -->
            {% if form.non_field_errors %}
              <div class="alert alert-danger">
                {{ form.non_field_errors }}
              </div>
            {% endif %}

            <!-- Campos con Bootstrap -->
            <div class="mb-3">
              <label for="id_nombre" class="form-label">Nombre</label>
              {{ form.nombre|add_class:"form-control" }}
              {{ form.nombre.errors }}
            </div>

            <div class="mb-3">
              <label for="id_apellido"
class="form-label">Apellido</label>
```

```

        {{ form.apellido|add_class:"form-control" }}
        {{ form.apellido.errors }}
    </div>

    <div class="mb-3">
        <label for="id_email" class="form-label">Correo
Electrónico</label>
        {{ form.email|add_class:"form-control" }}
        {{ form.email.errors }}
    </div>

    <div class="mb-3">
        <label for="id_profesion"
class="form-label">Profesión</label>
        {{ form.profesion|add_class:"form-control" }}
        {{ form.profesion.errors }}
    </div>

    <div class="d-grid">
        <button type="submit" class="btn btn-primary">
            <i class="bi bi-check-circle"></i> Guardar
        </button>
    </div>
</form>
</div>
</div>
</div>
</div>
</div>
{% endblock %}

```

En el template de profesores.html, vamos a agregar el botón para Agregar profesor por ejemplo debajo del encabezado:

```

<h1 class="text-center mb-4">Lista de Profesores</h1>
<div class="text-end mb-4"> <a class="btn btn-secondary" href="{%
url 'myapp:profesorFormulario' %}">Agregar Profesor</a></div>

```

Boton Agregar Profesor

Agregar Profesor

Formulario de Alta Profesor

Agregar Profesor

Nombre

Apellido

Correo Electrónico

Profesión

☒ Guardar

Como pueden ver en todo el proyecto, ya estamos incorporando estilos y componentes de bootstrap.

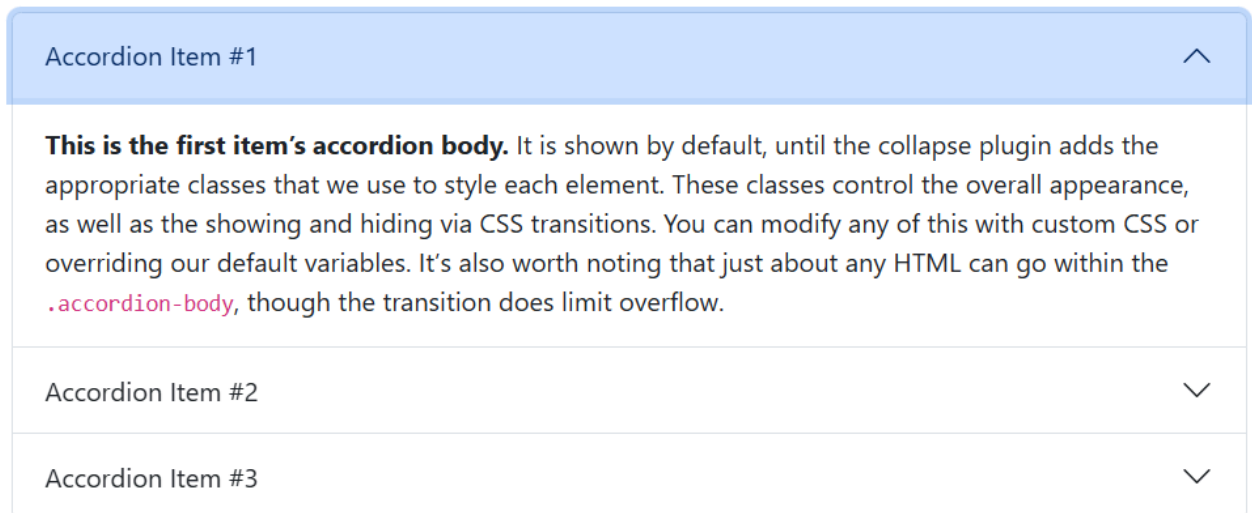
En su página oficial podemos ver más indicaciones de dar estilos a elementos html o también incorporar otras herramientas más como componentes.

La página oficial puedes visitar y ver todo lo que pueden mejorar de la vista de los html en:

[Accordion · Bootstrap v5.3](#)

Algunos componentes que se pueden ver son:

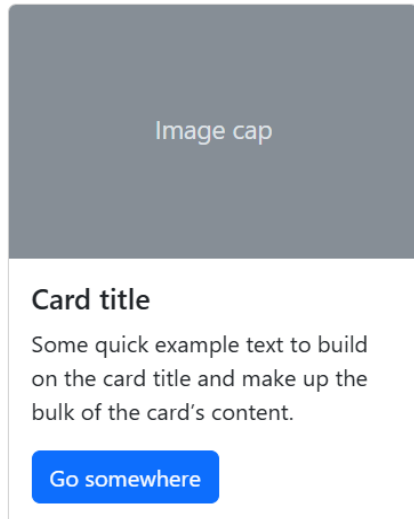
Acordeón



Botones



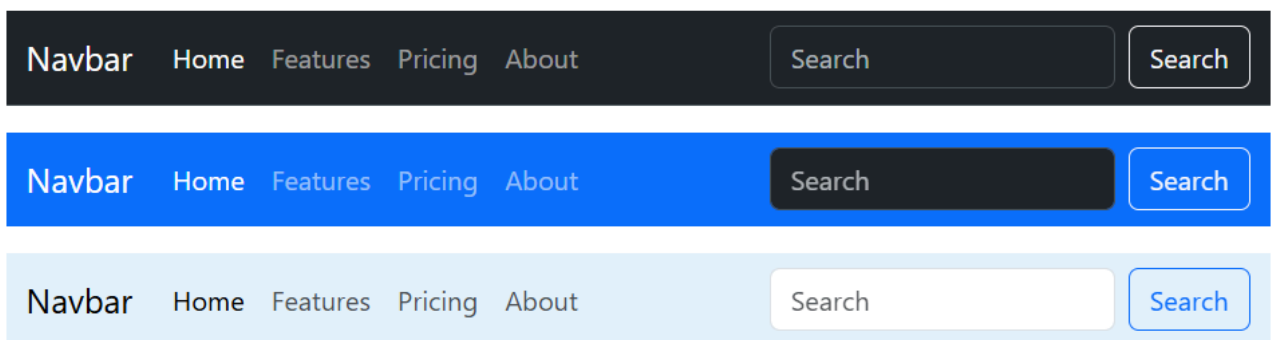
Tarjetas



Carrusel para imágenes



Menú de página



Instalación de bootstrap

El link de bootstrap dentro del head antes de tu css

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous">
  <link rel="stylesheet" href="./css/style.css">
  <title>Pruebas en clases</title>
</head>
```

El script dentro del body después del footer

```
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous">
</script>
</body>
```

Carrusel de imágenes

```
<div class="migrid">
  <div id="carouselExampleInterval" class="carousel slide" data-bs-ride="carousel">
    <div class="carousel-inner">
      <div class="carousel-item active" data-bs-interval="2000">
        
      </div>
      <div class="carousel-item" data-bs-interval="2000">
        
      </div>
      <div class="carousel-item">
        
      </div>
    </div>
    <button class="carousel-control-prev" type="button"
      data-bs-target="#carouselExampleInterval" data-bs-slide="prev">
      <span class="carousel-control-prev-icon" aria-hidden="true"></span>
      <span class="visually-hidden">Previous</span>
    </button>
    <button class="carousel-control-next" type="button"
      data-bs-target="#carouselExampleInterval" data-bs-slide="next">
      <span class="carousel-control-next-icon" aria-hidden="true"></span>
      <span class="visually-hidden">Next</span>
    </button>
  </div>
</div>
```

Navbar

```
<nav class="navbar bg-dark border-bottom sticky-top navbar-expand-md bg-body-tertiary" data-bs-theme="dark">
<div class="container-fluid">
  <a class="navbar-brand" href="#">Menu</a>
  <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
    aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarSupportedContent">
    <ul class="navbar-nav me-auto mb-2 mb-lg-0">
      <li class="nav-item">
        <a class="nav-link active" aria-current="page" href="#">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="#">Servicios</a>
      </li>
      <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown" aria-expanded="false">
          Productos
        </a>
        <ul class="dropdown-menu">
          <li><a class="dropdown-item" href="#">Consultar</a></li>
          <li><a class="dropdown-item" href="#">Vender</a></li>
          <li><hr class="dropdown-divider"></li>
          <li><a class="dropdown-item" href="#">Comprar</a></li>
        </ul>
      </li>
    </ul>
    <form class="d-flex" role="search">
      <input class="form-control me-2" type="search" placeholder="Search" aria-label="Search"/>
      <button class="btn btn-outline-success" type="submit">Search</button>
    </form>
  </div>
</div>
</nav>
```

Clases de bootstrap para tamaños

navbar-expand-lg

Breakpoint	Class infix	Dimensions
Extra small	<i>None</i>	<576px
Small	sm	≥576px
Medium	md	≥768px
Large	lg	≥992px
Extra large	xl	≥1200px
Extra extra large	xxl	≥1400px

Consulta de Profesores

Agregar la clase ProfesorForm en forms.py

```
from django import forms
from .models import Profesor

class CursoFormulario(forms.Form):
    nombre = forms.CharField(max_length=100)
    camada = forms.IntegerField()

class ProfesorFormulario(forms.Form):
    nombre = forms.CharField(max_length=100, label="Nombre")
    apellido = forms.CharField(max_length=100, label="Apellido")
    email = forms.EmailField(label="Correo Electrónico")
    profesion = forms.CharField(max_length=100, label="Profesión")

class ProfesorForm(forms.ModelForm):
    class Meta:
        model = Profesor
        fields = ['nombre', 'apellido', 'email', 'profesion']
```

Se deben realizar modificaciones en el views.py analizando si está importado models, si no, agregar al comienzo.

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Estudiante, Profesor, Curso, Entregable
from .forms import CursoFormulario, ProfesorFormulario, ProfesorForm
from django.db import models
```

Luego, reemplazar la vista de profesores por una que pregunte según lo que ingrese el usuario que pueda hacer la búsqueda, como el siguiente código:

```
def profesores(request):
    query = request.GET.get('q') # Captura lo que se escribe en el
    buscador
    if query:
        profesores = Profesor.objects.filter(
            models.Q(nombre__icontains=query) |
            models.Q(apellido__icontains=query) |
            models.Q(profesion__icontains=query)
        )
    else:
        profesores = Profesor.objects.all()
```

```
return render(request, 'myapp/profesores.html', {
    'profesores': profesores,
    'query': query,
})
```

- ♦ Aquí, lo que hacemos con `__icontains` es buscar sin distinguir mayúsculas/minúsculas.
- ♦ Y también usamos `models.Q` para combinar filtros con OR lógico (`|`).

Ahora debemos modificar `profesores.html` para agregar el buscador entre la lista de profesores y el botón de agregar profesor.

```
{% extends 'base.html' %}
{% block title %} Lista de Profesores {% endblock %}
{% block content %}
<div class="container mt-4">
    <h1 class="text-center mb-4">Lista de Profesores</h1>

    <!-- Botón para agregar -->
    <div class="text-end mb-4">
        <a class="btn btn-secondary" href="{% url
'myapp:profesorFormulario' %}">Agregar Profesor</a>
    </div>

    <!-- 🔍 Formulario de búsqueda -->
    <form method="get" action="{% url 'myapp:profesores' %}"
class="mb-4">
        <div class="input-group">
            <input type="text" name="q" class="form-control"
placeholder="Buscar por nombre, apellido o profesión"
            value="{{ query|default_if_none:'' }}">
            <button type="submit" class="btn btn-primary">
                <i class="bi bi-search"></i> Buscar
            </button>
        </div>
    </form>

    <!-- Resultados -->
    <div class="row">
        {% for profesor in profesores %}
        <div class="col-md-4 col-sm-6 mb-4">
            <div class="card shadow-sm h-100">
                <div class="card-body">
```

Lista de Profesores

Editor

Y con todo esto finalizamos la consulta de profesores. No olvidar siempre hacer push en su github para mantener su control de versiones.

Resumen para recordatorio de cómo utilizar git:

```
cd "C:\Ejercicios Python Aguirre\django_a_fondo"
git init
git status
git remote add origin https://github.com/usuario/mi_proyecto_django.git
git add .
git commit -m "Actualización: cambios locales"
git branch -M main
git pull origin main --rebase
git push origin main

    CADA VEZ QUE MODIFIQUES ALGO DEL PROYECTO ES:

git add .
git commit -m "Descripción breve de los cambios"
git push

    RECORDA HACERLO SIEMPRE EN LA CARPETA RAÍZ DEL PROYECTO
```

Modificación de Profesores

Ahora vamos a realizar las modificaciones, para ello agregaremos el botón de Editar en cada una de las tarjetas. Y crearemos el formulario de edición usando vistas basadas en funciones (FBV) o vistas genéricas (CBV).

Las vistas basadas en funciones son más didácticas para aprender, vamos a utilizar estas vistas.

Primero agregamos el botón de Editar dentro del for:

```
<div class="col-md-4 col-sm-6 mb-4">
    <div class="card shadow-sm h-100">
        <div class="card-body">
            <h5 class="card-title text-primary">{{
profesor.apellido }} {{ profesor.nombre }}</h5>
            <p class="card-text mb-1"><strong>Email:</strong> {{
profesor.email }}</p>
            <p class="card-text"><strong>Profesión:</strong> {{
profesor.profesion }}</p>
        </div>
        <div class="card-footer text-center">
            <a href="{% url 'myapp:profesorEditar' profesor.id %}"
class="btn btn-warning btn-sm me-2">Editar</a>
        </div>
```

```
</div>  
</div>
```

Aguirre Carlos Aníbal**Email:** carlosanibal815@gmail.com**Profesión:** Profesor Informática**Editar**

Agregar la url para editar en myapp/urls.py

```
path('profesor/editar/<int:id>/', views.profesor_editar,  
name='profesorEditar'),
```

Realizamos la vista para editar profesores en myapp/views.py

Primero analizamos lo que importamos, sería infaltable ProfesorForm del cual vamos a usar y models.

```
from django.shortcuts import render, redirect, get_object_or_404  
from .models import Estudiante, Profesor, Curso, Entregable  
from .forms import CursoFormulario, ProfesorFormulario, ProfesorForm  
from django.db import models
```

Luego sí, creamos las vistas profesor_editar:

```
def profesor_editar(request, id):  
    profesor = get_object_or_404(Profesor, id=id)  
  
    if request.method == 'POST':  
        form = ProfesorForm(request.POST, instance=profesor)  
        if form.is_valid():  
            form.save()  
            return redirect('myapp:profesores')  
    else:  
        form = ProfesorForm(instance=profesor)  
  
    return render(request, 'myapp/profesor_editar.html', {'form': form,  
    'profesor': profesor})
```

Template profesor_editar.html

Creá myapp/templates/myapp/profesor_editar.html:

```
{% extends 'base.html' %}
{% block title %}Editar Profesor{% endblock %}

{% block content %}
    <div class="container mt-4">
        <h2 class="text-center mb-4">Editar Profesor: {{ profesor.apellido
    }} {{ profesor.nombre }}</h2>

        <div class="card shadow-sm p-4 mx-auto" style="max-width: 600px;">
            <form method="POST">
                {% csrf_token %}
                {{ form.as_p }}

                <div class="text-center mt-4">
                    <button type="submit" class="btn btn-primary">Guardar
Cambios</button>
                    <a href="{% url 'myapp:profesores' %}" class="btn
btn-secondary ms-2">Cancelar</a>
                </div>
            </form>
        </div>
    </div>
{% endblock %}
```

Editar Profesor: Petris Raquel

Nombre:

Apellido:

Email:

Profesion:

© 2025 My Site. Todos los derechos reservados.

Baja de Profesores

Agregamos el botón de Eliminar dentro del for del html de profesores:

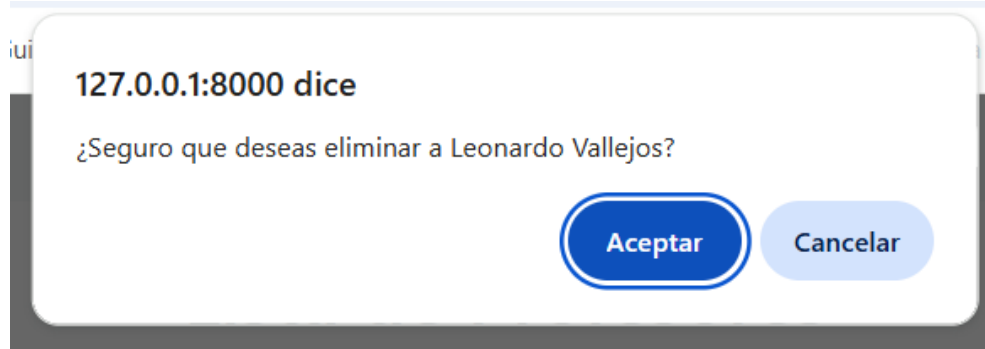
```
<div class="col-md-4 col-sm-6 mb-4">
  <div class="card shadow-sm h-100">
    <div class="card-body">
      <h5 class="card-title text-primary">{{
profesor.apellido }} {{ profesor.nombre }}</h5>
      <p class="card-text mb-1"><strong>Email:</strong> {{
profesor.email }}</p>
      <p class="card-text"><strong>Profesión:</strong> {{
profesor.profesion }}</p>
    </div>
    <div class="card-footer text-center">
      <a href="{% url 'myapp:profesorEditar' profesor.id %}"
class="btn btn-warning btn-sm me-2">Editar</a>
      <a href="{% url 'myapp:profesorEliminar' profesor.id
%}"
class="btn btn-danger btn-sm"
onclick="return confirm('¿Seguro que deseas eliminar a
{{ profesor.nombre }} {{ profesor.apellido }}?')">
Eliminar
</a>
    </div>
  </div>
</div>
```

Vista del botón de eliminar:

Lista de Profesores

[Agregar Profesor](#)[Buscar](#)**Aguirre Carlos Aníbal****Email:** carlosanibal815@gmail.com**Profesión:** Profesor Informática[Editar](#)[Eliminar](#)

Mensaje de advertencia antes de eliminar:



Creación del html profesor_confirm_delete.html

```
{% extends 'base.html' %}
{% block title %}Eliminar Profesor{% endblock %}
{% block content %}
    <div class="container mt-5 text-center">
        <h2 class="mb-4">¿Seguro que querés eliminar a {{ profesor.apellido }} {{ profesor.nombre }}?</h2>
        <form method="POST">
            {% csrf_token %}
            <button type="submit" class="btn btn-danger">Sí, eliminar</button>
            <a href="{% url 'myapp:profesores' %}" class="btn btn-secondary ms-2">Cancelar</a>
        </form>
    </div>
{% endblock %}
```

```
</div>
{% endblock %}
```

Template de advertencia opcional donde se confirma para eliminar:

¿Seguro que querés eliminar a Vallejos Leonardo?

Sí, eliminar

Cancelar

© 2025 My Site. Todos los derechos reservados.

Incorporando más funcionalidades:

- ☰ Nueva app para autenticación y perfiles - Proyecto Cursos
- ☰ Cambiando la presentación - Proyecto Cursos